

Predicting Marathon Race Performance Using Machine Learning

Project Progress Update #2

**Enhanced Evaluation, Error Interpretation, and Decision-
Based Prediction**

IECE 465: Introduction to Machine Learning for Engineers

Faculty Mentor: Dr. Hany Elgala

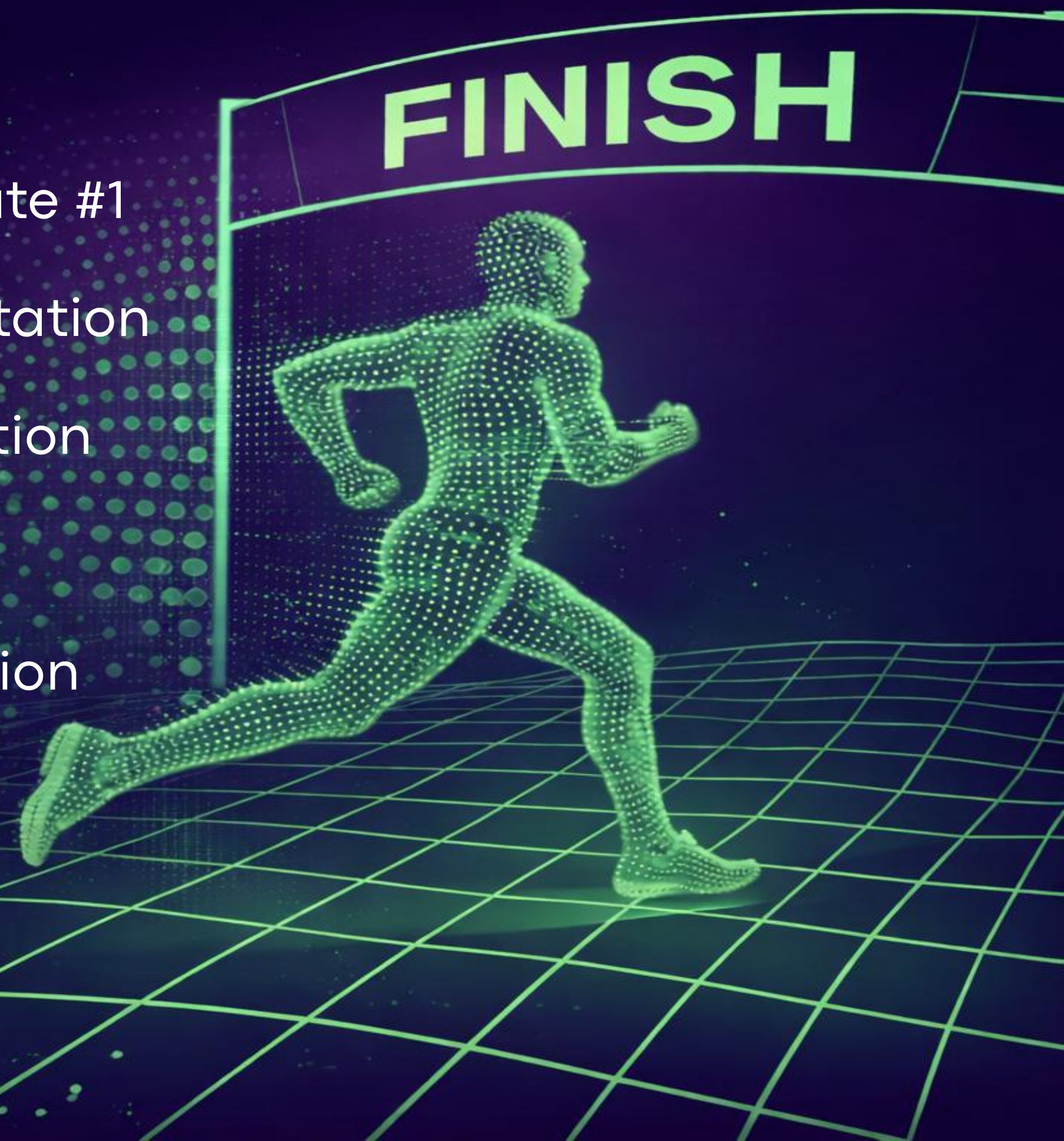
Teaching Assistant: Honglan Chen

Team Members: Sanad Sahawneh, Brian Allen, and Mahib Rahman

Date: Monday, April 13th, 2026 / Wednesday, April 15th, 2026

AGENDA

1. Project Recap
2. Key Improvements Since Project Update #1
3. System Pipeline and Model Implementation
4. Evaluation Framework and Interpretation
5. Results and Visual Analysis
6. Sports Betting Decision-Based Extension
7. Code Walkthrough
8. Team Contributions
9. Live Demonstration



BRIEF PROJECT RECAP

Objective: Predict marathon finish time using machine learning models applied to real-world race data.

Dataset:

- 2023 Boston Marathon (~26,598 runners)

```
data = readtable('boston_marathon_2023.csv');  
data = rmmissing(data);
```

Input Features:

- Age group
- Gender
- Half-marathon split time

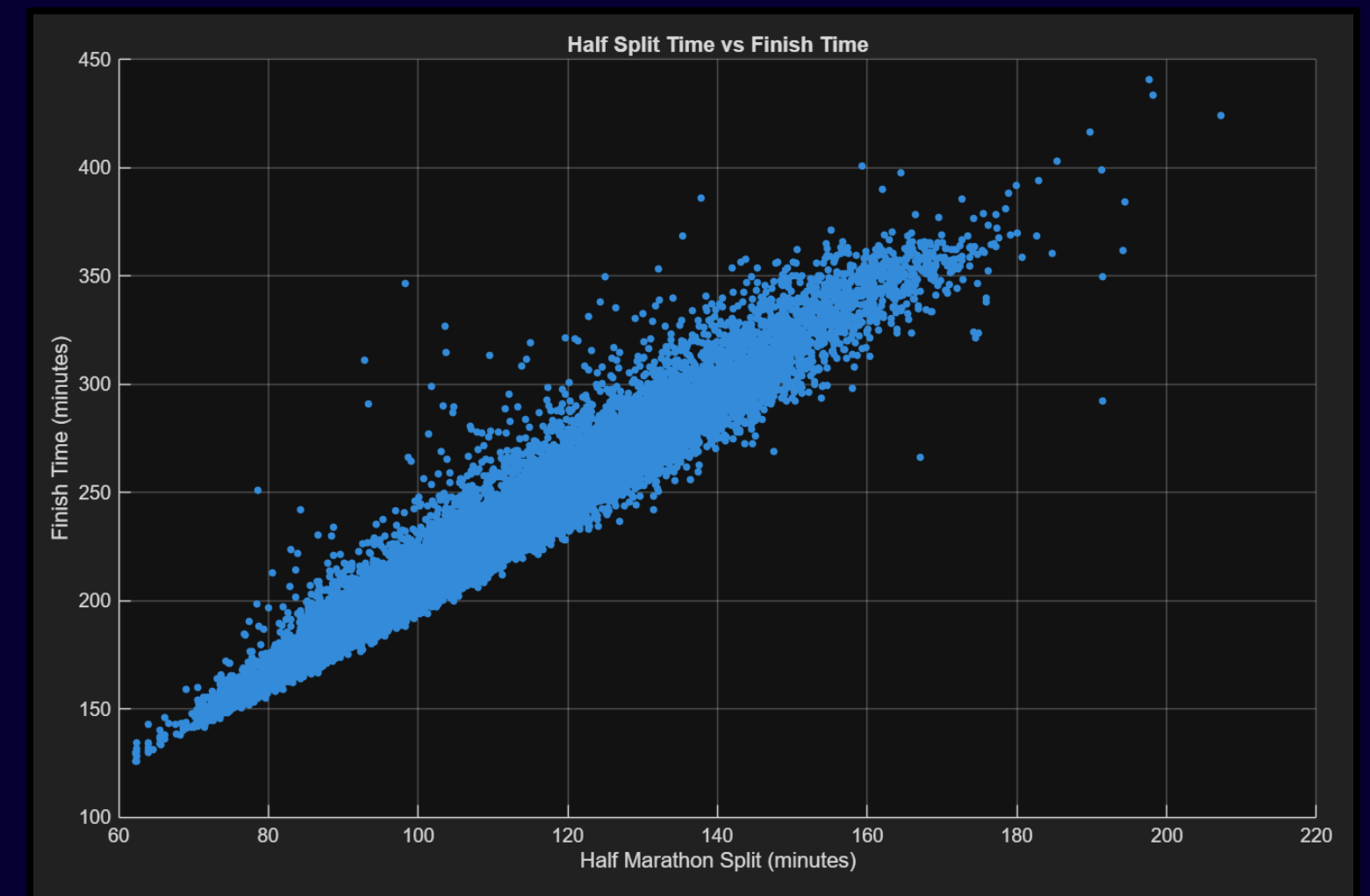
Target:

- Final net finish time (seconds)

Models (Update #1):

- Linear Regression
- Neural Network

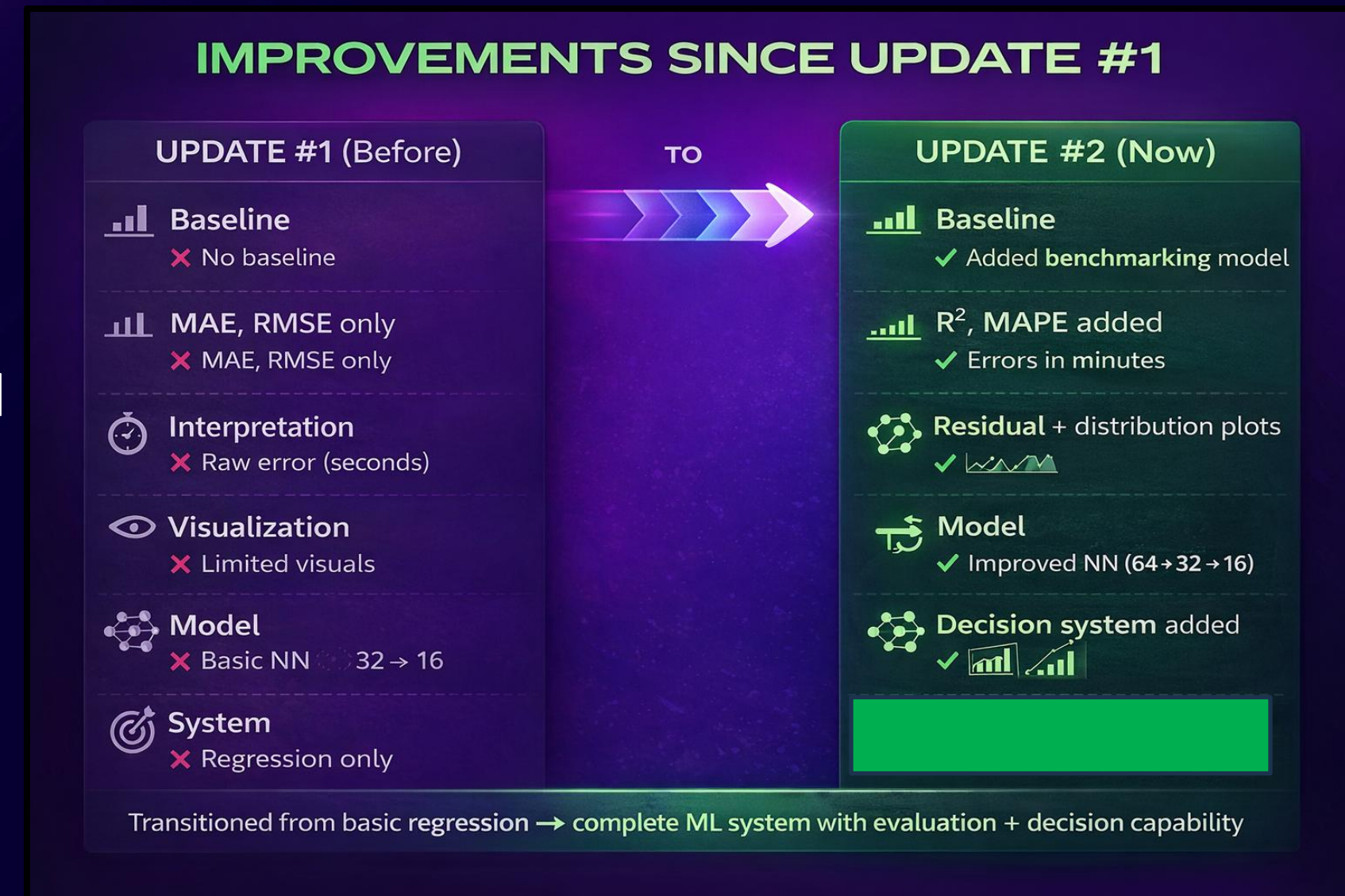
Strong linear relationship motivates regression-based modeling.



KEY IMPROVEMENTS SINCE UPDATE #1

Major Achievements:

- Introduced baseline model for meaningful benchmarking
- Expanded evaluation metrics:
From MAE, RMSE → to MAE, RMSE, R², MAPE
- Converted prediction error into minutes for real-world interpretation
- Developed prediction quality classification framework
- Implemented residual analysis and error distribution visualization
- Improved neural network architecture (64 → 32 → 16)
- Added sports betting style over/under prediction system
- Built full end-to-end ML pipeline with extensive visualization



This update transitions the project from basic modeling to a complete ML system with evaluation and decision capability.

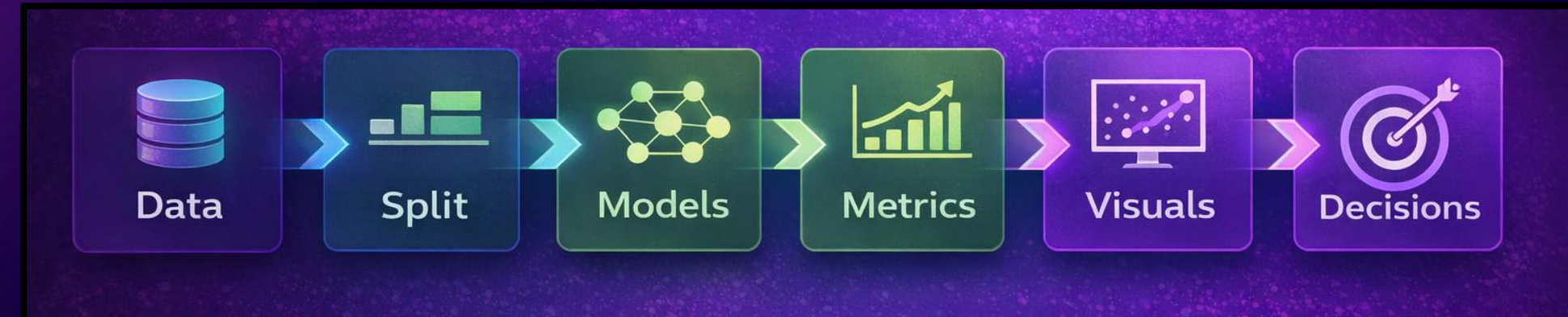
END-TO-END ML PIPELINE

Why this Pipeline Matters?

- Prevents overfitting, ensures fair evaluation, and enables reproducibility.

Pipeline Stages:

- **Data** → Load and clean marathon dataset
- **Split** → Create training and testing sets (80/20)
- **Models** → Train baseline, linear, and neural models
- **Metrics** → Evaluate using MAE, RMSE, R^2 , MAPE
- **Visuals** → Analyze predictions and residuals
- **Decisions** → Convert predictions into over/under outcomes



```
cv = cvpartition(height(data), 'HoldOut', 0.20);  
trainData = data(training(cv), :);  
testData = data(test(cv), :);
```

This code creates an 80/20 train-test split to ensure unbiased model evaluation.

BASELINE + LINEAR MODEL

Baseline Model:

- Predicts finish time using average ratio
- Establishes minimum expected performance

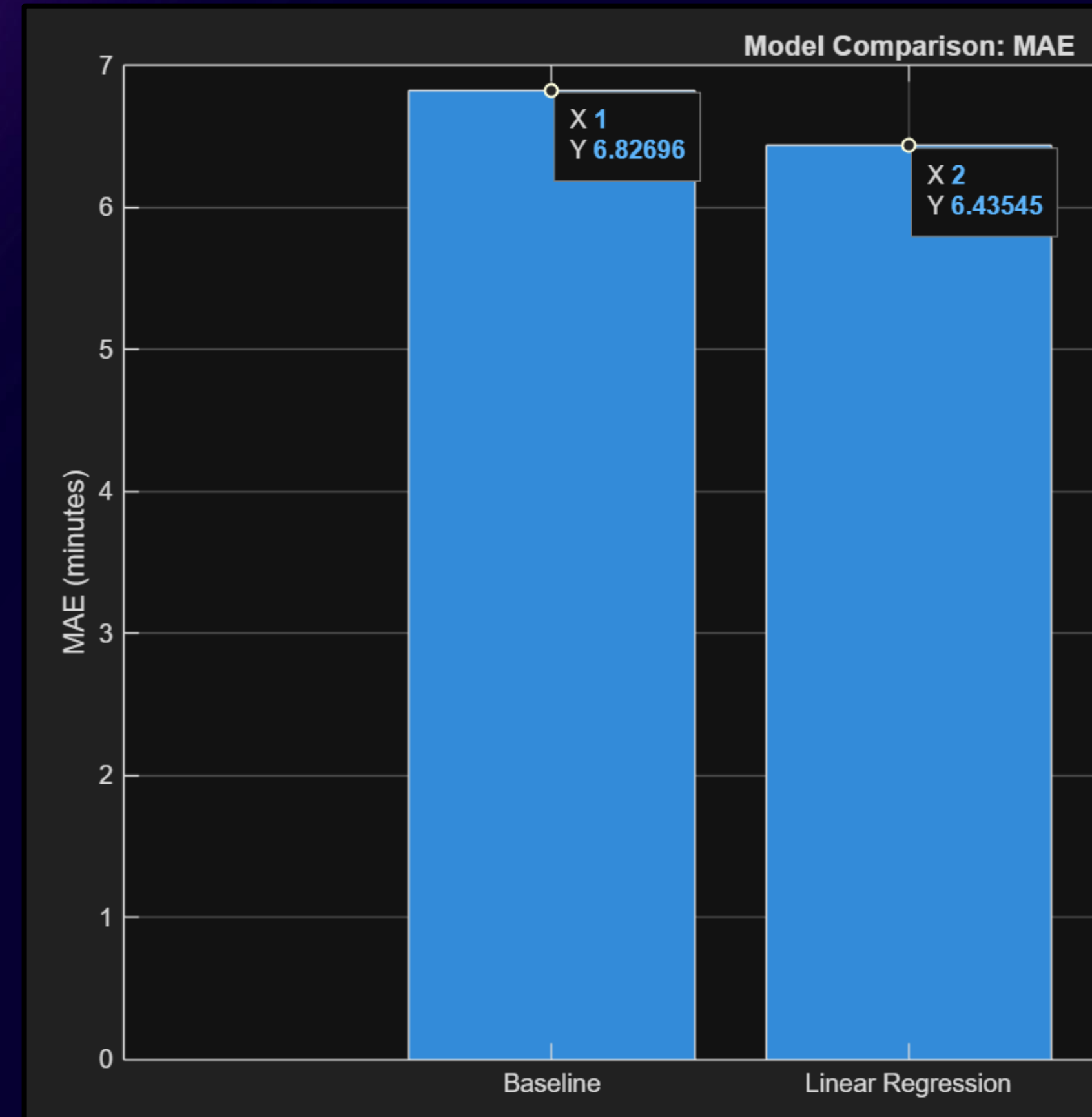
Linear Regression:

- Interpretable model
- Strong performance due to dominant feature

```
avgRatio = mean(trainData.finish_net_sec ./ trainData.half_time_sec);  
yPred_base = testData.half_time_sec .* avgRatio;
```

```
linModel = fitlm(trainData, ...  
'finish_net_sec ~ age_group + gender + half_time_sec');
```

We compute a baseline prediction using average ratios, then train a linear model using key features.



Linear model reduces error relative to baseline.

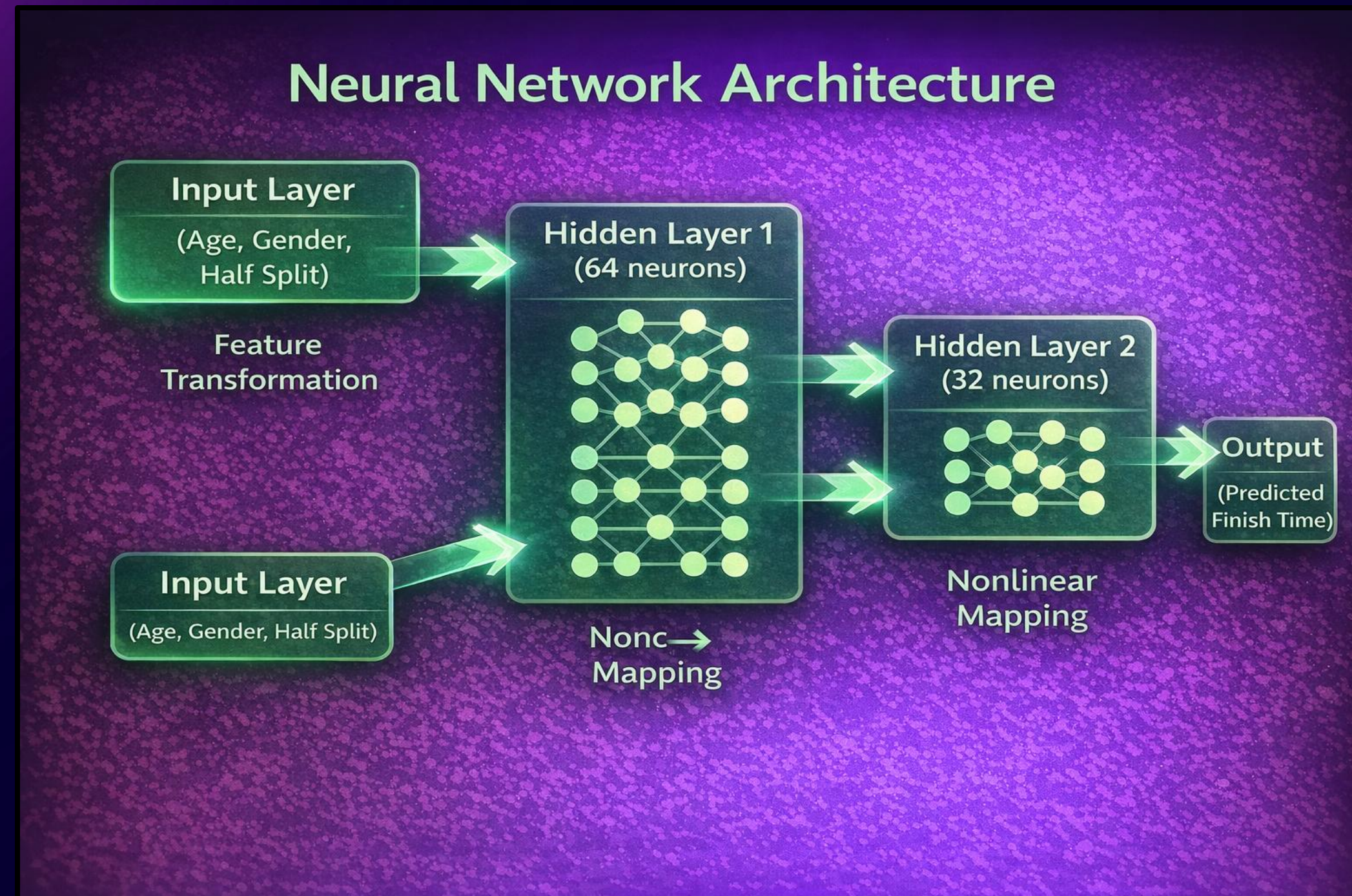
NEURAL NETWORK

Neural Network Model:

- Multi-Layer Perceptron
- Architecture: $64 \rightarrow 32 \rightarrow 16$
- Captures nonlinear relationships
- Used standardized inputs

```
nnModel = fitrnet(Xtrain_std, yTrain, ...  
'LayerSizes', [64 32 16], ...  
'Activations', 'relu');
```

This trains a multi-layer neural network to capture nonlinear relationships in the data.



Provides slight improvement over linear model by capturing nonlinear patterns.

EVALUATION FRAMEWORK

Mean Absolute Error (MAE):

$$\text{MAE} = (1/n) * \sum |y - \hat{y}|$$

- Measures average prediction error
- Interpretable in original units (seconds → minutes)

Root Mean Squared Error (RMSE):

$$\text{RMSE} = \text{sqrt}((1/n) * \sum (y - \hat{y})^2)$$

- Penalizes larger errors more heavily
- Sensitive to outliers

Coefficient of Determination (R^2):

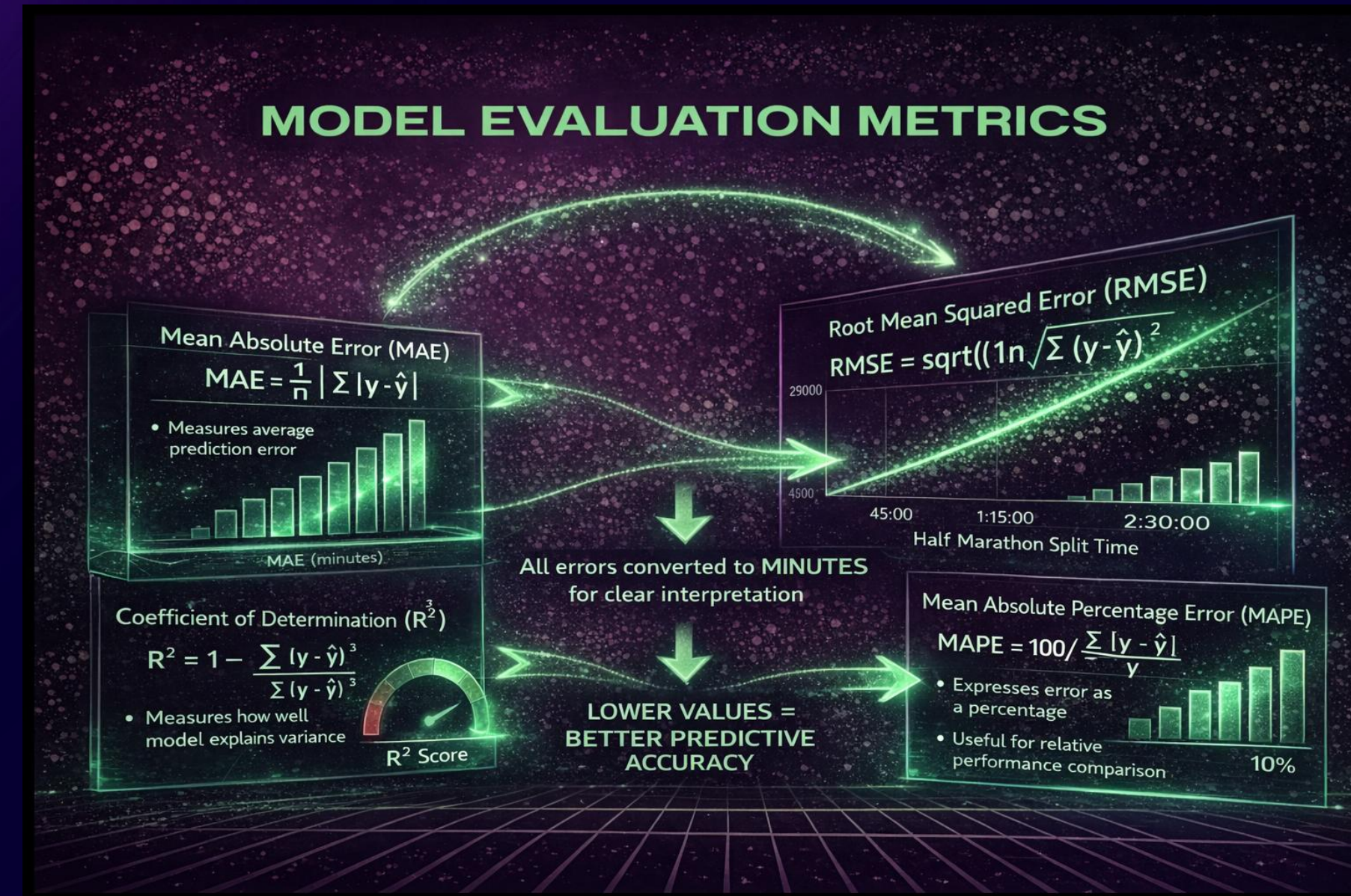
$$R^2 = 1 - (\sum (y - \hat{y})^2 / \sum (y - \bar{y})^2)$$

- Measures how well the model explains variance
- Values closer to 1 indicate better fit

Mean Absolute Percentage Error (MAPE):

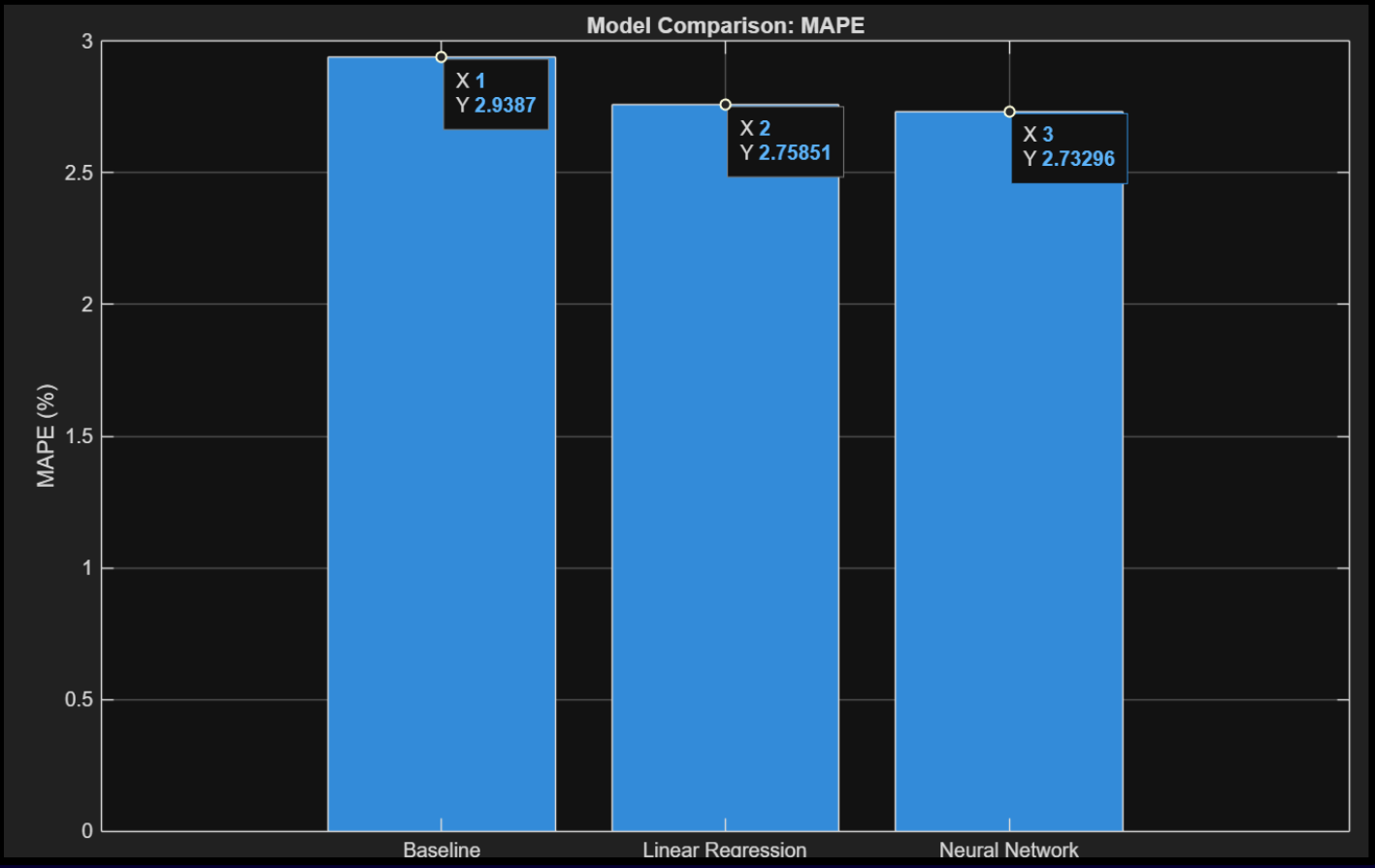
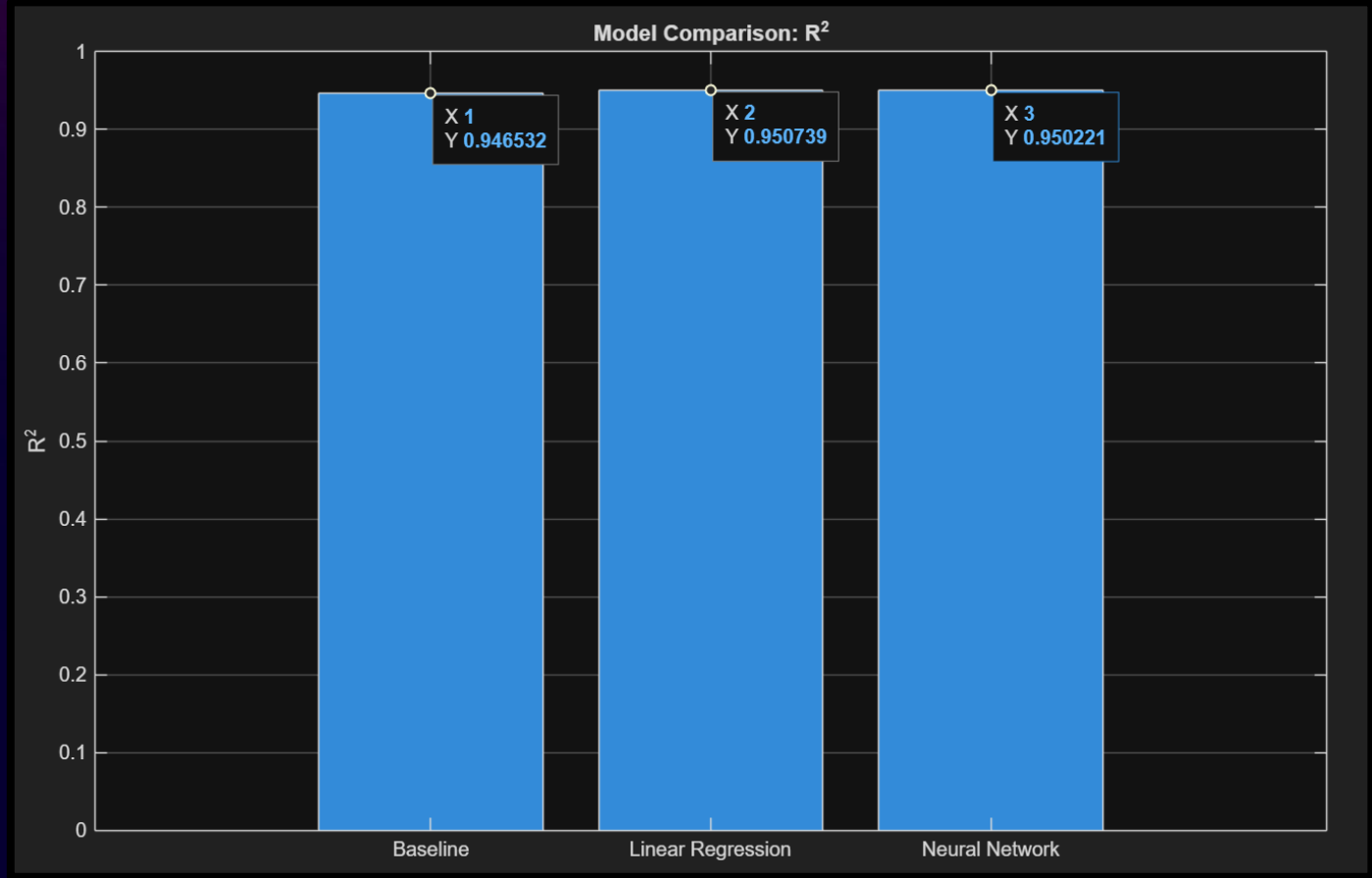
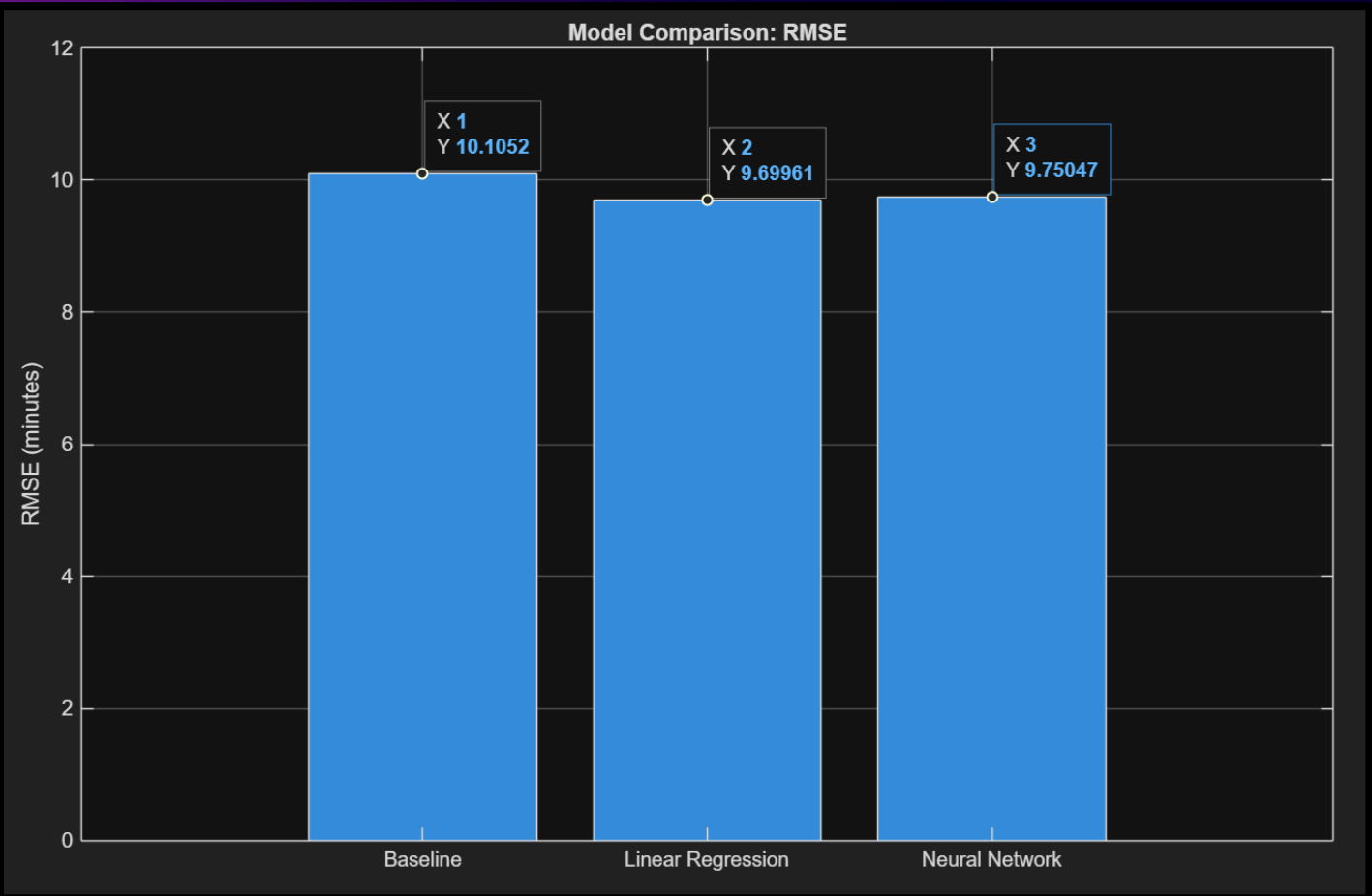
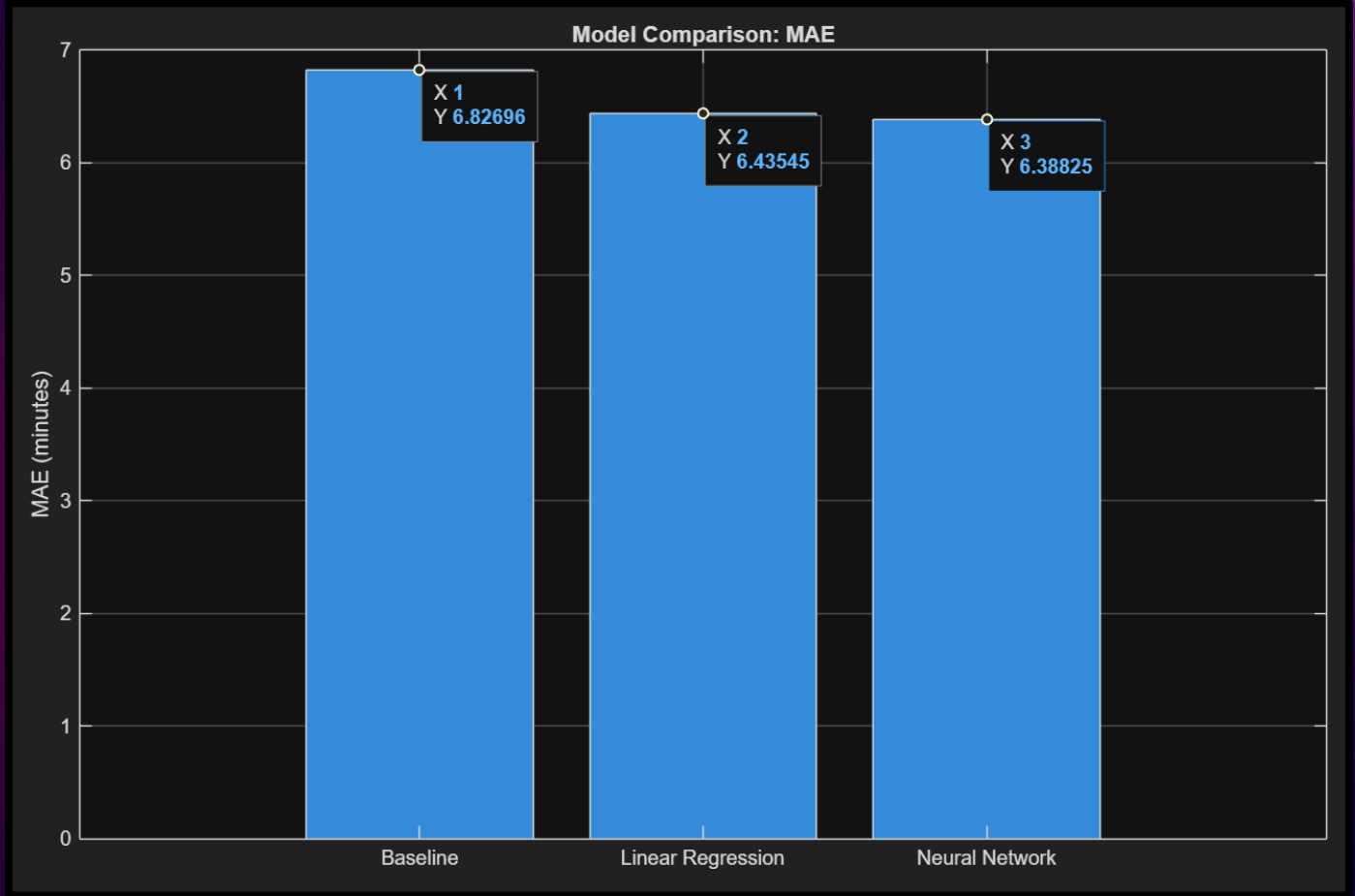
$$\text{MAPE} = (100/n) * \sum |(y - \hat{y}) / y|$$

- Expresses error as a percentage
- Useful for relative performance comparison



These metrics quantify model accuracy and allow comparison between models.

EVALUATION METRIC PERFORMANCE COMPARISON



Observations:

- Neural network achieves lowest error
- Linear model performs similarly
- Baseline provides useful benchmark

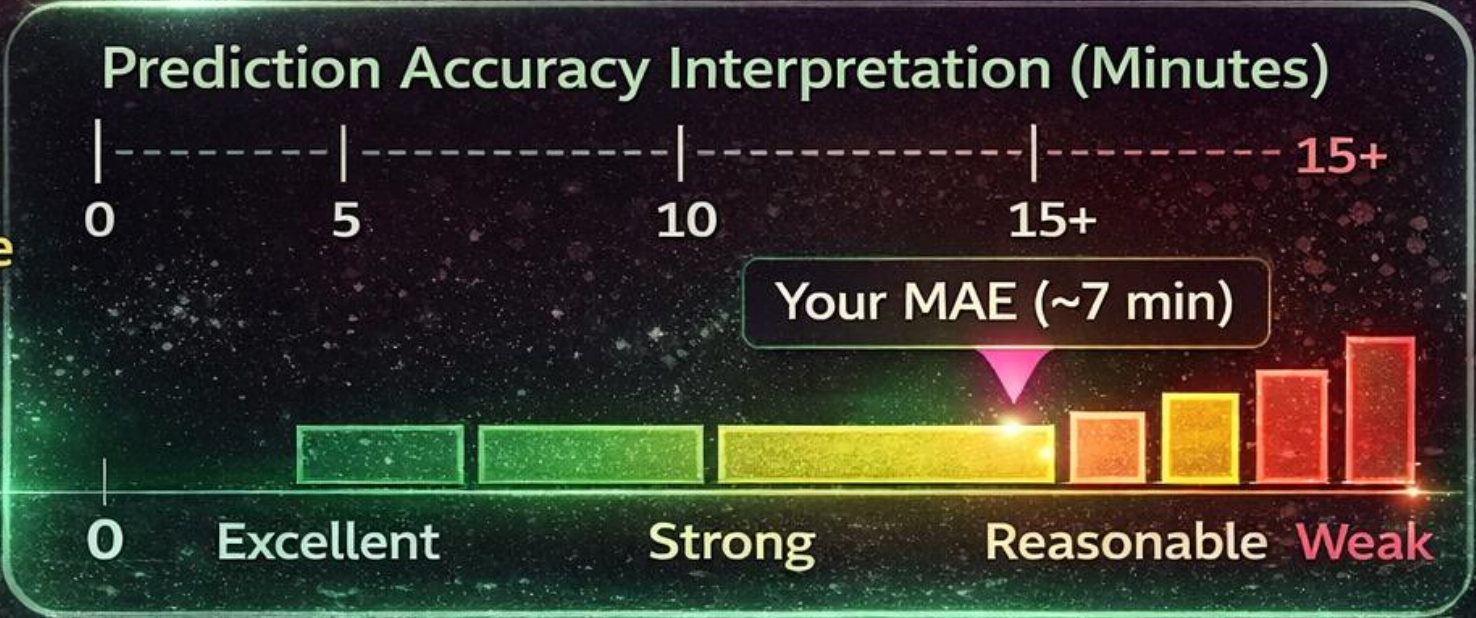
INTERPRETING MODEL PERFORMANCE



Interpreting Model Error (Real-World Meaning)

Prediction Quality (in Minutes)

- < 5 minutes → **Excellent**
 - Very accurate prediction
- 5–10 minutes → **Strong**
 - Reliable for practical use
- 10–15 minutes → **Reasonable**
 - Acceptable but less precise
- > 15 minutes → **Weak**
 - Limited predictive reliability



===== MODEL RESULTS =====						
Model	MAE_sec	RMSE_sec	MAE_min	RMSE_min	MAPE_pct	R2_score
"Baseline"	409.62	606.31	6.827	10.105	2.9387	0.94653
"Linear Regression"	386.13	581.98	6.4354	9.6996	2.7585	0.95074
"Neural Network"	383.3	585.03	6.3883	9.7505	2.733	0.95022

RESULTS

Model Prediction Visualization:

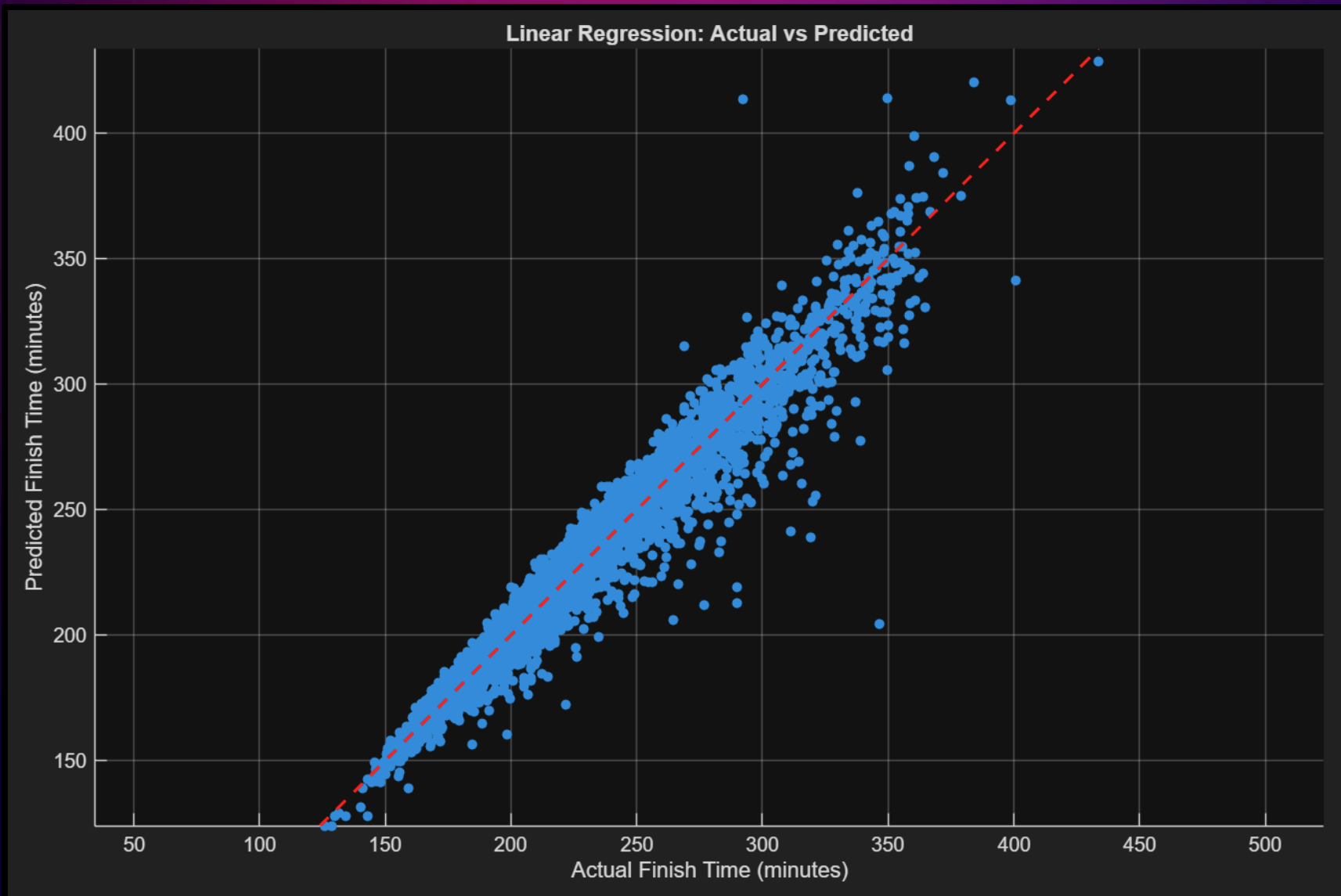
% Linear Regression

```
scatter(yTest/60, y_pred_lin/60, 'filled'); hold on  
plot([min max]/60, [min max]/60, 'r--')
```

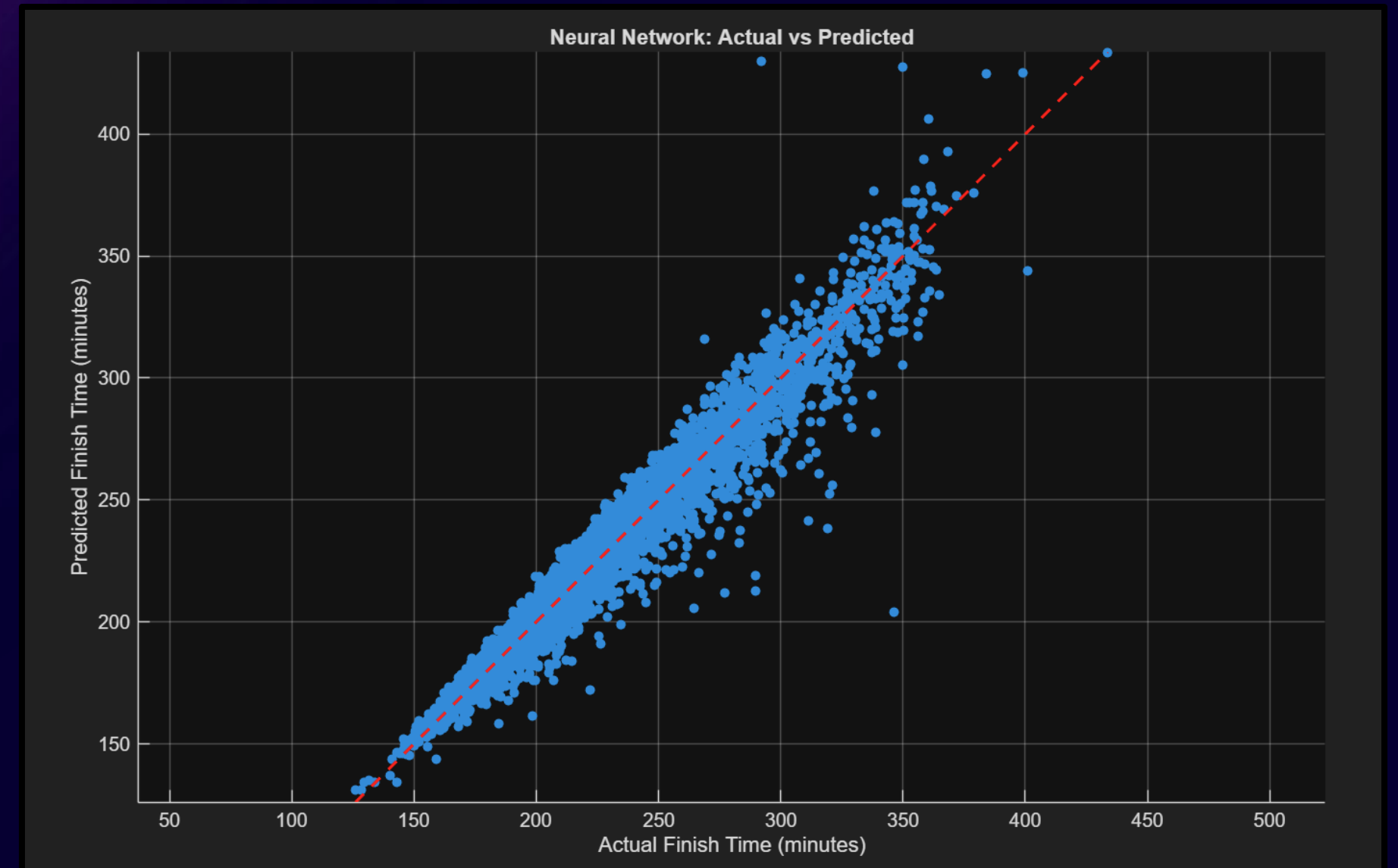
% Neural Network

```
scatter(yTest/60, y_pred_nn/60, 'filled'); hold on  
plot([min max]/60, [min max]/60, 'r--')
```

Linear Regression Prediction Plot



Neural Network Prediction Plot



Interpretation:

- Points close to diagonal line = accurate predictions.
- Both models show strong predictive capability.

VISUAL ANALYSIS: RESIDUALS

What are Residuals? → Residual = Actual – Predicted

- Measure prediction error for each sample
- Ideal model → residuals centered around 0

```
res_lin = yTest - yPred_lin;
```

```
res_nn = yTest - yPred_nn;
```

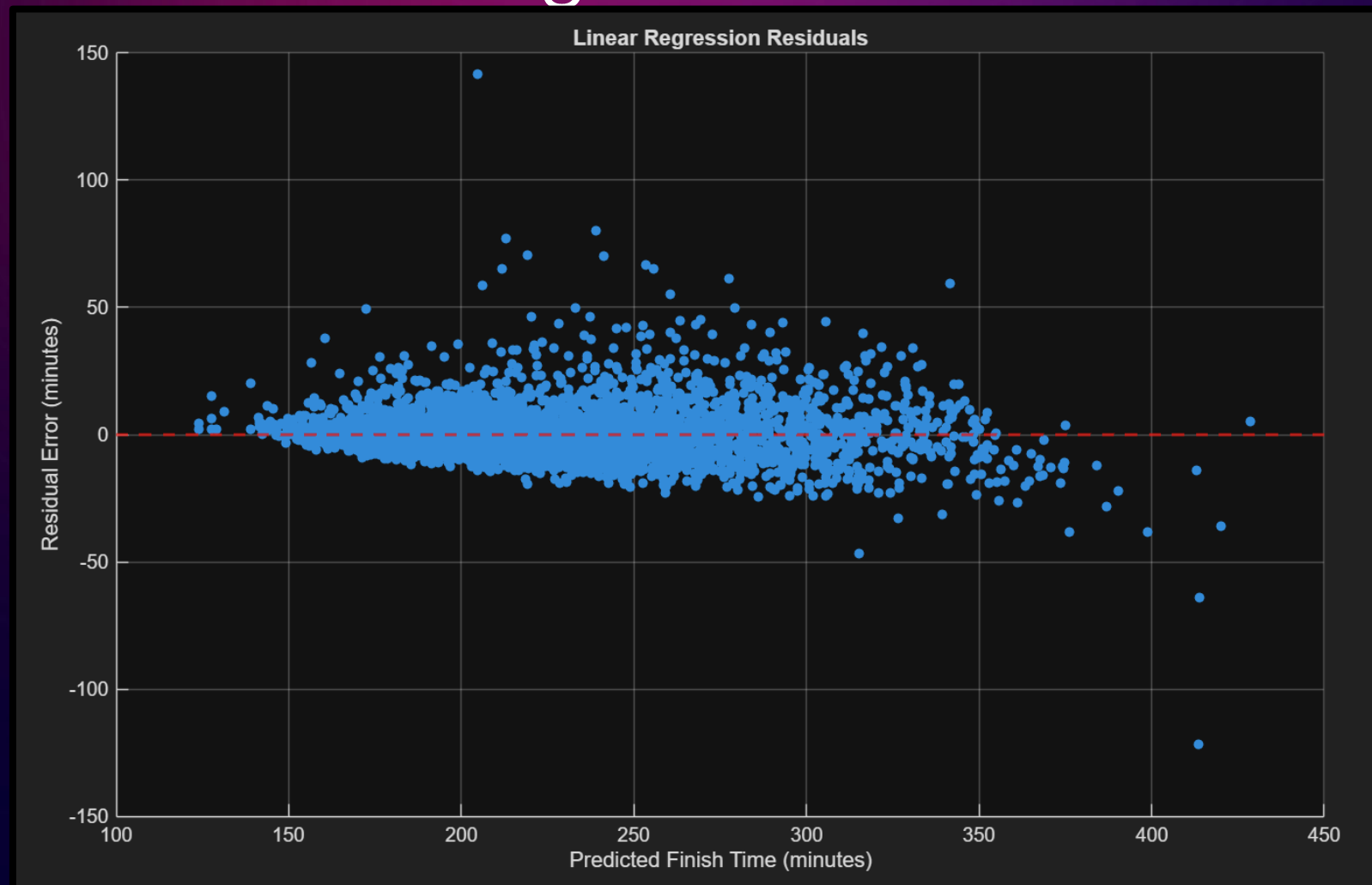
```
scatter(yPred_lin/60, res_lin/60, 'filled'); hold on;
```

```
ylines(0,'r--')
```

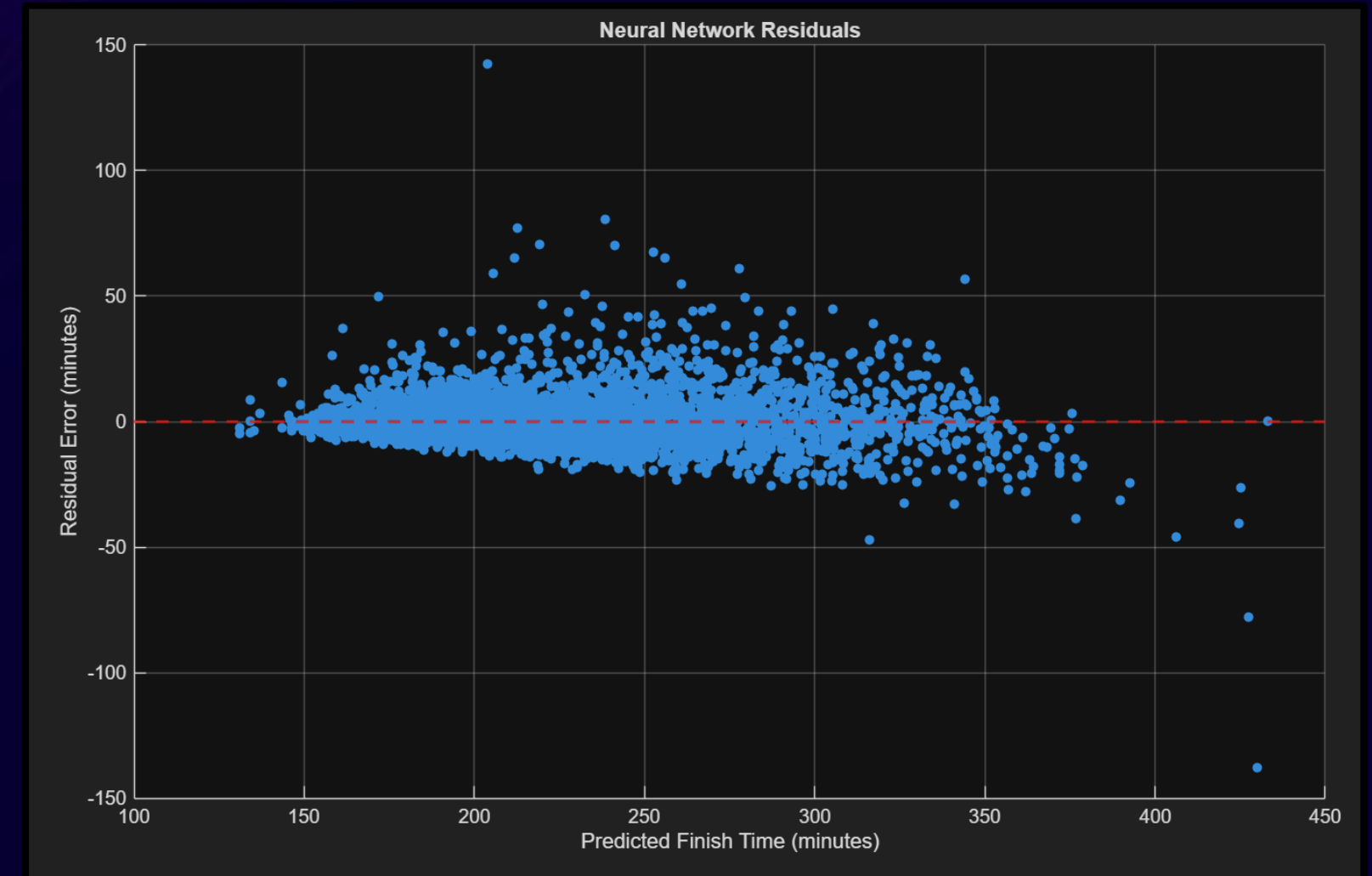
```
scatter(yPred_nn/60, res_nn/60, 'filled'); hold
```

```
on; ylines(0,'r--')
```

Linear Regression Residuals



Neural Network Residuals



Key Observations:

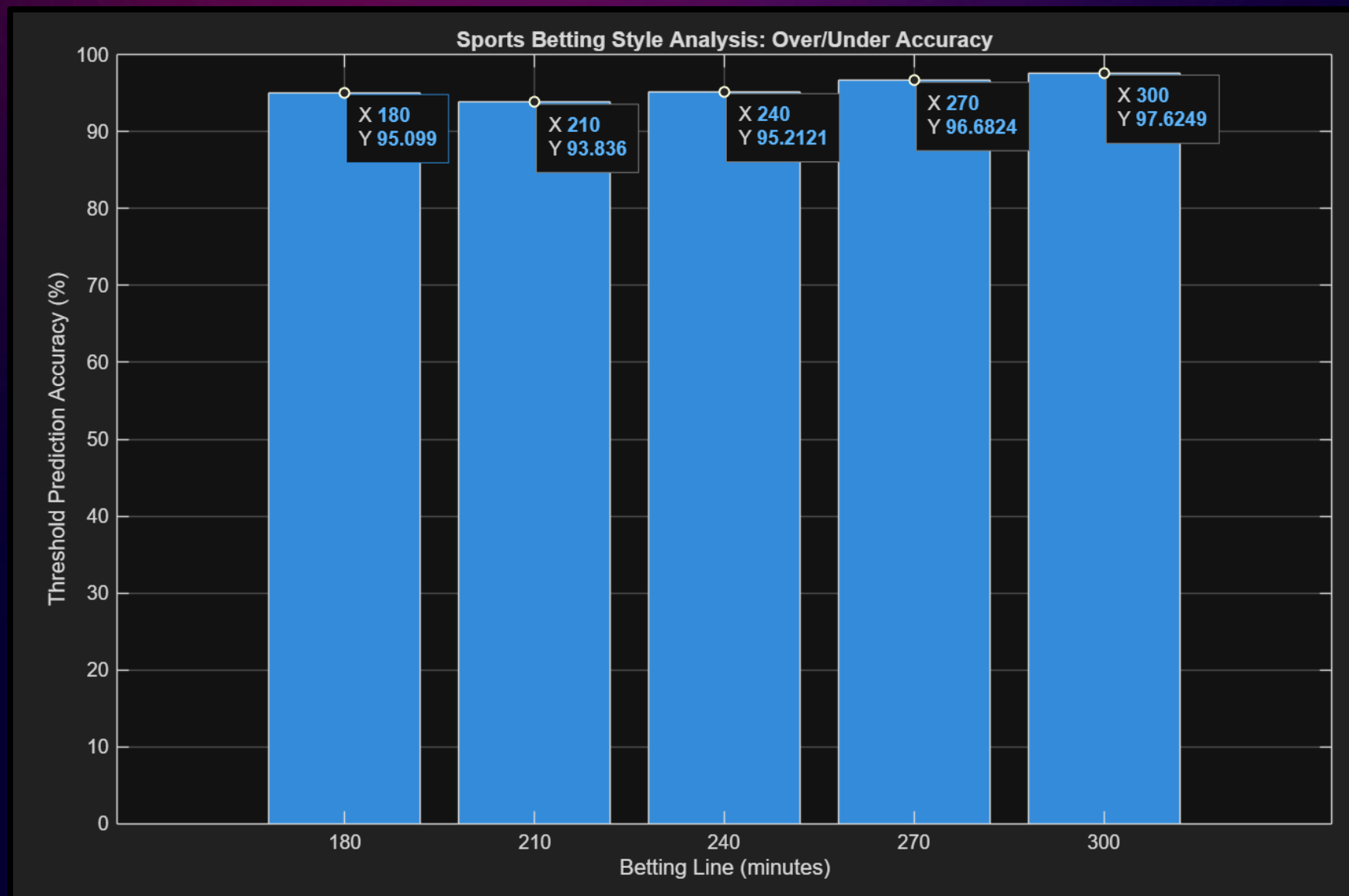
- Residuals are **centered around zero** → low bias. No strong pattern → model captures main relationship
- Slight spread increase at higher times → harder predictions for slower runners
- Neural network shows **slightly tighter spread** → marginal improvement

DECISION SYSTEM: OVER/UNDER PREDICTION

Concept:

- Convert regression output → binary decision
- Predict if runner finishes UNDER threshold or OVER threshold
- Thresholds represent realistic race targets

Decision Accuracy



```
betLines_min = [180 240 300];
```

```
predUnder = yPred < line_sec;
```

```
actualUnder = yTest < line_sec;
```

```
accuracy = mean(predUnder == actualUnder);
```

This code converts predicted times into over/under decisions and computes accuracy.

Key Observations:

- High accuracy across all thresholds (~94–97%)
- Model reliably predicts race outcomes
- Slight variation depending on threshold difficulty

Transforms ML model from prediction → actionable decision system with strong real-world applicability.

SYSTEM DEMONSTRATION

Live Demo Steps:

- Run training script
- Display model results
- Show prediction plots
- Show over/under decision output

TEAM CONTRIBUTIONS



Sanad Sahawneh



Designed and implemented full ML **pipeline**



Developed evaluation framework (MAE, RMSE, R^2 , MAPE)



Built decision-based prediction system (**over/under**)



Led **visual analysis** and results interpretation



Brian Allen



Made the **original code skeleton** of the ML models



Data preprocessing and cleaning



Feature **selection** and dataset preparation



Implemented **linear regression** and **neural network** models



Mahib Rahman



Feature **encoding** and **normalization**



Model **tuning** and **optimization**



Performance evaluation **support & validation**



Takeaway

End-to-end **system** development integrating **data**, **modeling**, and **decision-making**



NEXT STEPS

Turning Predictions into Impact



Enhance Model & Features

- Add external features (weather, pacing, elevation)
- Identify key predictors (feature importance)



Expand Decision System

- Extend over/under thresholds
- Explore classification-based models



Strengthen Evaluation

- Analyze performance across groups (age, gender)
- Investigate large errors and outliers



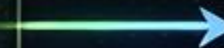
Move Toward Deployment

- Build prediction interface
- Enable real-time predictions
- Support real-world decision making

Current System

Enhanced Model

Real-World Decision Support



Questions?

